

π Skill

π Skill论文笔记

论文: [\$\pi\$ Skill: High-Density Knowledge Extraction from Single Trajectories via Circular Step-Level Analysis](#) (ACL ARR 2026 May 投稿, 当前公开稿修改于 2026-07-03, 尚在匿名评审)

作者: Chenxi Qiu、Zichen Luo、Jiarui Fan、Keming Gu、Jiaqi Zhang、Chunbo Zhang、Peng Zhang (作者顺序由[作者主页](#)交叉核验; PDF 本身匿名)

机构: 当前匿名稿未披露, 无法按论文署名可靠核验

代码: 未找到可核验的公开 π Skill 实现; 可参考其前身 [XSkill-Agent/XSkill](#), 但它不是本文新增 CSD/CMU 的完整实现

数据: [VisualToolBench](#)、[TIR-Bench](#)、[MMSearch-Plus](#)、[MM-BrowseComp](#)、[AgentVista](#)

Problem

多模态智能体已经能够调用图像分析、代码执行和网页搜索等工具完成复杂任务, 但通常存在一个问题: **执行过的任务不会自然转化为可以持续复用的知识**

在不更新模型参数的情况下, 能否从极少量、甚至单条智能体轨迹中, 提取出可靠、细粒度且可复用的经验, 使智能体在后续任务中持续改进

Gap (现有研究的缺陷、痛点)

现有训练免费型记忆与技能学习方法主要存在三个问题:

- **轨迹总结过于粗糙, 粒度不够** 很多方法直接总结完整轨迹, 容易丢失具体在哪一步成功、在哪一步失败, 以及中间出现了什么关键转折

AWM (Agent Workflow Memory) :

每个workflow固定为

Workflow Description(工作流目标描述)

Workflow Trajectory(步骤序列)

$p_i = (\text{环境状态描述}, \text{推理}, \text{动作})$

成功轨迹中的错误也可能被保留, 失败轨迹中的正确步骤会被一起丢弃, 动作执行的后果

- **知识缺少可靠性管理。** 经验通常以静态 Markdown 或 JSON 保存, 没有置信度、版本、生命周期和回滚机制, 错误知识可能长期污染后续推理

Reasoning bank保存: 原任务+原推理动作轨迹+成功/失败标签+提炼出的策略条目

仅仅保存一次运行成功/失败标签不足以判断该条经验是否可靠

- **检索与执行彼此割裂。** 检索通常只依赖语义相似度, 不区分经验是否可靠, 也很少根据后续执行结果更新经验

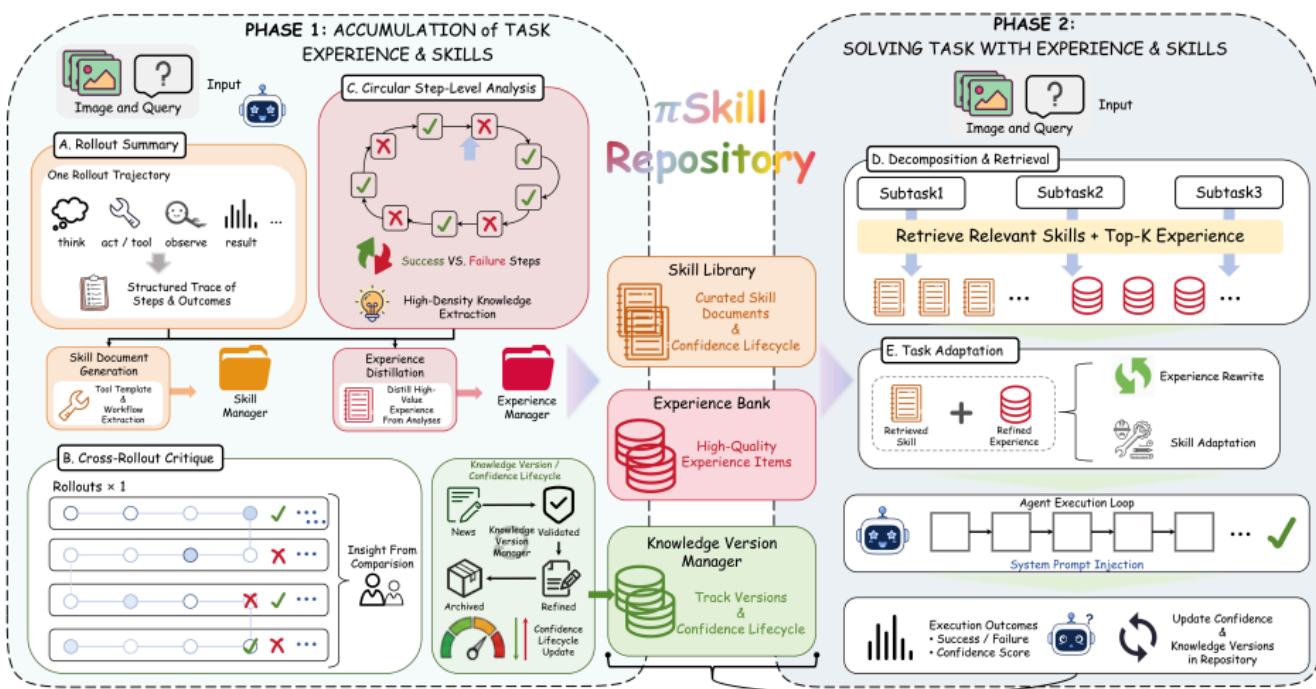
Core idea（文章的核心思想、insight）

核心思想：不要只总结一条轨迹最终成功或失败的原因，而要分析其中每一个状态—动作—新状态的转移，把局部经验提取为可独立管理的 Experience Atom

每个Experience Atom包含：

- 适用场景与领域
- 成功或失败的局部教训
- 可复用的工具调用模板
- 成功次数
- 失败次数
- 动态置信度

How（文章核心方法如何设计）



πSkill主要由两个模块组成

Phase 1 CSD

1. 生成多条执行轨迹

输入：问题+图片+可用工具集合

输出：包含状态、动作和工具返回结果的轨迹

智能体对同一个任务执行多次 rollout：

2. CSD：循环式步骤级知识蒸馏

- 对每条轨迹中的每个局部转移(s_t, a_t, s_{t+1})进行评价，根据最终任务结果，分析哪些局部动作有效、无效或存在风险
输入：原始任务X+动作前状态 s_t +当前动作 a_t +动作后状态 s_{t+1} +最终结果 Outcome(τ)
输出：大模型当前步骤的局部批评
包括：
识别成功转移识别失败点
定位具体负责成功或失败的子步骤
总结局部动作
对最终结果的贡献
- 对齐同一任务的成功当前步骤的局部批评轨迹和失败轨迹，寻找两者的关键分叉点
输入：同一题的成功轨迹+失败轨迹
输出：关键分叉节点及其成功—失败差异
- 将这些局部发现提取为 Experience Atoms
输入：单轨迹步骤批评+关键分叉节点及其成功—失败差异
输出：experience atom
 - c：置信度
 - domain：适用任务范围
 - lesson：自然语言经验
 - tool_template：抽象工具操作格式
 - ϕ_{succ} ：成功次数
 - ϕ_{fail} ：失败次数

Phase 2 CMU

1. experience与skill建立关联

输入：experience和skill

处理之后每个experience与多个skill建立关联

2. 置信度与状态更新

输入	含义
c_{prior}	更新前的置信度
ϕ_{succ}	该 Experience 的成功记录数
ϕ_{fail}	该 Experience 的失败记录数
status_prior	原生命周期状态
执行结果	新轨迹最终成功或失败
α	历史置信度的衰减/融合系数
ϵ	防止分母为零的平滑项

statusprior: Candidate、Active、Archived 根据更新后的置信度判断更新
置信度计算公式:

$$c_{\text{new}} = \alpha c_{\text{prior}} + (1-\alpha) \frac{\phi_{\text{succ}}}{\phi_{\text{succ}} + \phi_{\text{fail}} + \epsilon}$$

Phase 3 在新任务中检索、改写并使用知识

1.任务分解

输入: 问题+图片

输出: 拆分出的多个子问题

2.选择experience

粗召回:

输入: e_{g_i} : 子问题的向量表示 e_E : 经验的向量表示

输出: 前k个experience

置信度过滤与冲突处理:

输入: 前k个experience和子问题

处理:

过滤掉置信度较低的经验

处理冲突experience:

置信度更高的 Experience;

或者领域结构匹配得更具体的 Experience

输出: 经过滤和消除冲突的experience集合

3.写experience并适配skill

1.经验改写:

- 输入: 经过过滤和冲突处理的 experience E 、当前任务 X_{test}
- 处理:
 - 删除只与原训练任务有关的具体信息
 - 保留可复用的经验
 - 根据当前任务的输入与约束重新表述经验

- 输出:
即适用于当前任务的 experience E

2.Skill 适配:

- 输入:
 - 筛选出的 experience 所关联的 skill 指针
 - Skill Base 中的抽象技能模板
 - 当前任务输入 X_{test}
- 处理:
 - SkillBuilder 根据 Experience-Skill 映射找到相关 skill
 - adaptation engine 根据当前任务调整具体执行模板

- 输出：
 - 改写后的 experience E
 - 适用于当前任务的 skill 工具调用模板

4. Agent 执行与在线反馈

1. Agent 执行：

- 输入：
 - 当前任务 X_{test}
 - 改写后的 experience E
 - 适配后的 skill 和工具调用模板
- 处理：
 - 将 E 和适配后的 skill 注入 Agent 上下文
 - Agent 根据经验和技能规划操作
 - Agent 调用外部工具执行任务
- 输出：
 - 新的执行轨迹 τ_{eval}
 - 可验证的最终结果：Success 或 Failure

2. 在线反馈：

- 输入：
 - 执行轨迹 τ_{eval}
 - 最终成功或失败结果
- 处理：
 - 执行成功：增加 ϕ_{succ}
 - 执行失败：增加 ϕ_{fail}
 - 根据新计数重新计算置信度
 - 更新 experience 的生命周期状态
 - 将更新结果写回 Experience Database
- 输出：
 - 更新后的成功/失败次数
 - 更新后的置信度
 - 更新后的 Candidate、Active 或 Archived 状态
 - 更新后的 Experience Database 和知识库版本

形成记忆更新闭环：

Experience Database → 检索与过滤 → 经验改写与 Skill 适配 → Agent 执行 → Success/Failure → 更

Evidence（实验、证据、消融）

banckbone：Qwen2.5-VL-7B-Instruct

数据集：VisualToolBench、TIR-Bench、MMSearch-Plus 、AgentVista

每个任务执行4次

两个主要指标是：

- Average@4：四次执行的平均成功表现
- Pass@4：四次中至少成功一次的比例

Benchmark	Average@4		Pass@4	
	XSkill	π Skill	XSkill	π Skill
VisualToolBench	9.11	11.33+2.22	21.03	25.23+4.20
TIR-Bench	28.12	29.13+1.01	58.50	60.00+1.50
MMSearch-Plus	4.62	6.00+1.38	11.00	13.50+2.50
AgentVista	7.80	9.63+1.83	15.60	20.18+4.58
Avg	12.41	14.02+1.61	26.53	29.73+3.20

消融实验：

- 只加入 CSD：12.12 / 25.49，低于 XSkill
- 只加入 CMU：12.24 / 26.39，也没有稳定超过 XSkill
- 同时加入 CSD 和 CMU：14.02 / 29.73，才取得明显提升

Methods	VisualToolBench		TIR-Bench		MMSearch-Plus		AgentVista		Avg	
	Average@4	Pass@4	Average@4	Pass@4	Average@4	Pass@4	Average@4	Pass@4	Average@4	Pass@4
XSkill	9.11	21.03	28.12	58.50	4.62	11.00	7.80	15.60	12.41	26.53
XSkill+CSD	10.28+1.17	22.43+1.40	25.38-2.74	51.50-7.00	5.00+0.38	11.50+0.50	7.80+0.00	16.51+0.91	12.12-0.29	25.49-1.04
XSkill+CMU	10.63+1.52	24.77+3.74	25.75-2.37	51.50-7.00	3.87-0.75	10.00-1.00	8.72+0.92	19.27+3.67	12.24-0.17	26.39-0.14
π Skill	11.33+2.22	25.23+4.20	29.13+1.01	60.00+1.50	6.00+1.38	13.50+2.50	9.63+1.83	20.18+4.58	14.02+1.61	29.73+3.20

这说明两个模块可能存在协同作用，不能独立证明每个模块本身都有效

My Takeway

1.多经验协同下，如何完成因果信用分配？

一次任务同时检索了多个 Experience，并调用了一个或多个 Skill，但系统最终只得到一个 Success/Failure。如何判断成功或失败究竟由哪条 Experience、哪个 Skill 或哪个步骤造成，从而只更新真正相关的知识？

2.经验真正可靠还是调用次数频繁？

高置信度 Experience 更容易被检索，而被检索得越多，它获得成功记录的机会也越多；低置信

度 Candidate 则可能长期得不到验证。如何区分“经验本身可靠”和“经验只是被频繁使用”，并在利用已有经验与探索新经验之间取得平衡？

低频但关键的安全经验会不会被归档？如何确保这类经验安全存储

3.成功轨迹和失败轨迹应该各生成多少条？

简单任务很难得到失败轨迹，困难任务很难得到成功轨迹。系统能否根据当前不确定性，自适应决定继续采样成功轨迹、失败轨迹，还是停止 rollout